

Database Management System

Unit-III

A. Hemalatha Reddy
Assistant Professor
CSE-Data Science

Part-A: Basics of SQL, DDL, DML, DCL, structure - creation, alteration, defining constraints - Primary key, foreign key, unique, not null, check, in operator, Functions - aggregate functions, built-in functions - numeric, date, string functions, set operations.

Introduction to SQL

SQL stands for **Structured Query Language**.

It is used to communicate with relational databases.

SQL can be used to:

1. Create databases and tables
2. Insert and modify data
3. Retrieve data
4. Control user access
5. Apply constraints and rules

Examples: MySQL, Oracle, SQL Server, PostgreSQL

SQL commands are mainly classified into:

Category	Meaning	Examples
DDL	Data Definition Language	CREATE, ALTER, DROP
DML	Data Manipulation Language	INSERT, UPDATE, DELETE
DCL	Data Control Language	GRANT, REVOKE
DQL	Data Query Language	SELECT
TCL	Transaction Control Language	COMMIT, ROLLBACK

SQL Functions

SQL provides functions for performing calculations and manipulating data.

Two major categories:

1. Aggregate Functions

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

2. Built-in Functions

- Numeric functions
- Date functions
- String functions

Aggregate Functions

Function	Purpose
COUNT()	Counts records
SUM()	Calculates total
AVG()	Calculates average
MAX()	Finds maximum
MIN()	Finds minimum

1. Count()

It is used to count the number of rows in a table. It helps summarize data by giving the total number of entries. It can be used in different ways depending on what you want to count:

COUNT(*): Counts all rows.

COUNT(column_name): Counts non-NULL values in the specified column.

COUNT(DISTINCT column_name): Counts unique non-NULL values in the column.

2. SUM()

It is used to calculate the total of a numeric column. It adds up all non-NULL values in that column. For Example, SUM(column_name) returns sum of all non-NULL values in the specified column.

Query:

-- Calculate the total salary

```
SELECT SUM(Salary) AS TotalSalary FROM Employee;
```

-- Calculate the sum of unique salaries

```
SELECT SUM(DISTINCT Salary) AS DistinctSalarySum FROM Employee;
```

3. AVG()

It is used to calculate average value of a numeric column. It divides sum of all non-NULL values by the number of non-NULL rows. For Example, AVG(column_name) returns average of all non-NULL values in the specified column.

Query:

-- Calculate the average salary

```
SELECT AVG(Salary) AS AverageSalary FROM Employee;
```

-- Average of distinct salaries

```
SELECT AVG(DISTINCT Salary) AS DistinctAvgSalary FROM Employee;
```


MIN() and MAX()

The MIN() and MAX() functions return the smallest and largest values, respectively, from a column.

Query:

-- Find the highest salary

```
SELECT MAX(Salary) AS HighestSalary FROM Employee;
```

-- Find the lowest salary

```
SELECT MIN(Salary) AS LowestSalary FROM Employee;
```

Date Functions in SQL

SQL Date Functions are built-in tools used to handle, modify and analyze date/time values in a database. They help perform tasks like retrieving current dates, calculating differences and formatting results effectively.

- Extract specific parts of a date (year, month, day).
- Format dates for user-friendly display.
- Track trends, deadlines or schedules in business processes.

Let's use the sales table as the base for demonstrating each of the SQL Date Functions mentioned. Here's the structure of the sales table:

Numeric Functions in SQL: Numeric functions are used to perform mathematical calculations on numeric data.

Function	Description	Example	Output
ABS()	Returns absolute value	ABS(-10)	10
CEIL() / CEILING()	Rounds up to nearest integer	CEIL(4.2)	5
FLOOR()	Rounds down	FLOOR(4.8)	4
ROUND()	Rounds a number	ROUND(15.678, 2)	15.68
POWER()	Calculates power	POWER(2,3)	8
MOD()	Returns remainder	MOD(10,3)	1
SQRT()	Returns square root	SQRT(25)	5
SIGN()	Returns sign of number	SIGN(-20)	-1

Ex: SELECT ABS(-25), ROUND(15.678, 2), MOD(10, 3);

Date Functions: Date functions are used to work with dates and times in a database.

Function	Description	Example
CURRENT_DATE	Returns current date	CURRENT_DATE
CURRENT_TIMESTAMP	Returns current date and time	CURRENT_TIMESTAMP
YEAR()	Extracts year	YEAR('2026-08-17') → 2026
MONTH()	Extracts month	MONTH('2026-08-17') → 8
DAY()	Extracts day	DAY('2026-08-17') → 17
DATE_ADD()	Adds time to a date	DATE_ADD(date, INTERVAL 5 DAY)
DATE_SUB()	Subtracts time from a date	DATE_SUB(date, INTERVAL 5 DAY)

EX: SELECT CURRENT_DATE;

O/P: 2026-08-17

EX: SELECT DATEDIFF('2026-08-20', '2026-08-17');

O/P: 3

Function	Description	Example	Output
UPPER()	Converts a string to uppercase	UPPER('hello')	HELLO
LOWER()	Converts a string to lowercase	LOWER('HELLO')	hello
LENGTH()	Returns the number of characters	LENGTH('Database')	8
SUBSTRING() or SUBSTR()	Extracts part of a string	SUBSTRING('Database', 1, 4)	Data
CONCAT()	Joins two or more strings	CONCAT('Data', 'Base')	DataBase
TRIM()	Removes leading and trailing spaces	TRIM(' Hello ')	Hello
LTRIM()	Removes leading spaces	LTRIM(' Hello')	Hello
RTRIM()	Removes trailing spaces	RTRIM('Hello ')	Hello
REPLACE()	Replaces part of a string	REPLACE('apple', 'p', 'b')	abble
INSTR()	Finds the position of a substring	INSTR('Database', 'base')	5
LEFT()	Returns leftmost characters	LEFT('Database', 4)	Data
RIGHT()	Returns rightmost characters	RIGHT('Database', 4)	base
REVERSE()	Reverses a string	REVERSE('SQL')	LQS

String Functions

```
EX: SELECT UPPER(name), LOWER(name),  
      LENGTH(name), CONCAT(first_name, ' ', last_name),  
      SUBSTRING(name, 1, 3)  
FROM Student;
```

Set Operators: Set operations are used to combine the results of two or more **SELECT** queries.

Operator	Purpose
UNION	Combines results and removes duplicates
UNION ALL	Combines results and keeps duplicates
INTERSECT	Returns common rows
EXCEPT / MINUS	Returns rows in first query but not second

UNION:

Combines results from two queries and **removes duplicates**.

Ex: SELECT city FROM Students UNION SELECT city FROM Teachers;

UNION ALL:

Combines results but **keeps duplicates**.

Ex: SELECT city FROM Students UNION ALL SELECT city FROM Teachers;

INTERSECT:

Returns only the rows that are **common to both queries**.

Ex: SELECT city FROM Students INTERSECT SELECT city FROM Teachers;

EXCEPT / MINUS:

Returns rows present in the **first query but not in the second**.

Ex: SELECT city FROM Students EXCEPT SELECT city FROM Teachers;

Thank you